

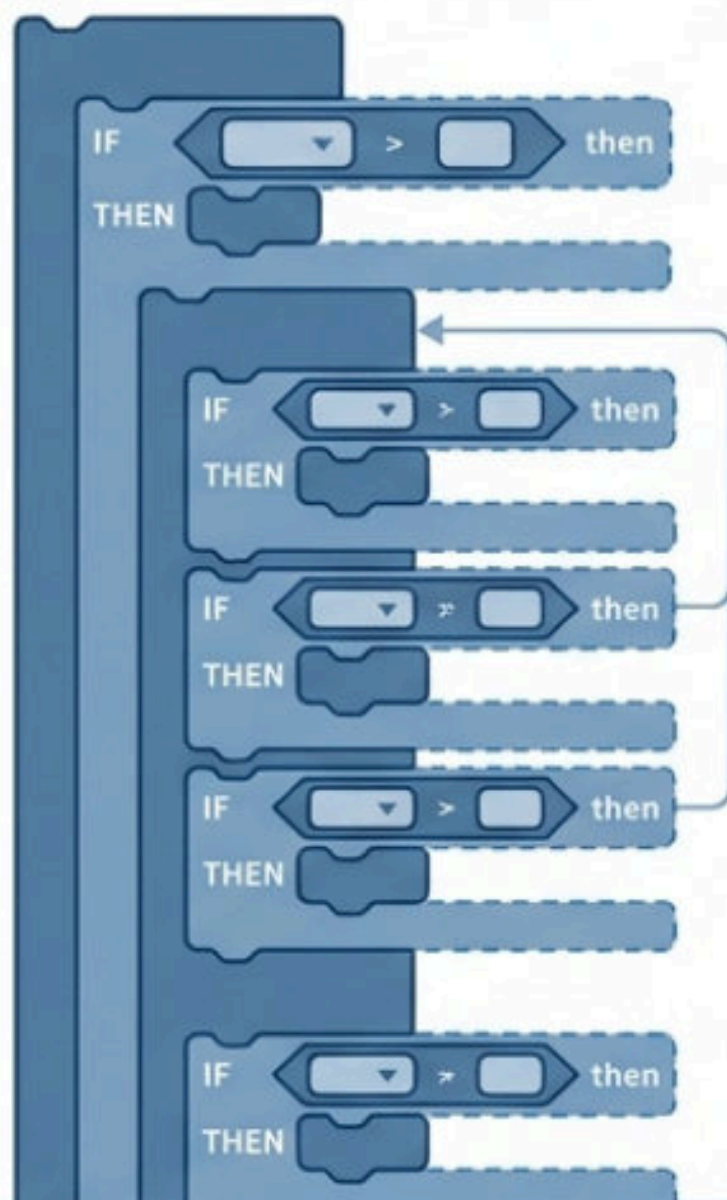
分類器測試基地：Scratch 互動與 AI 邊界



啟動測試

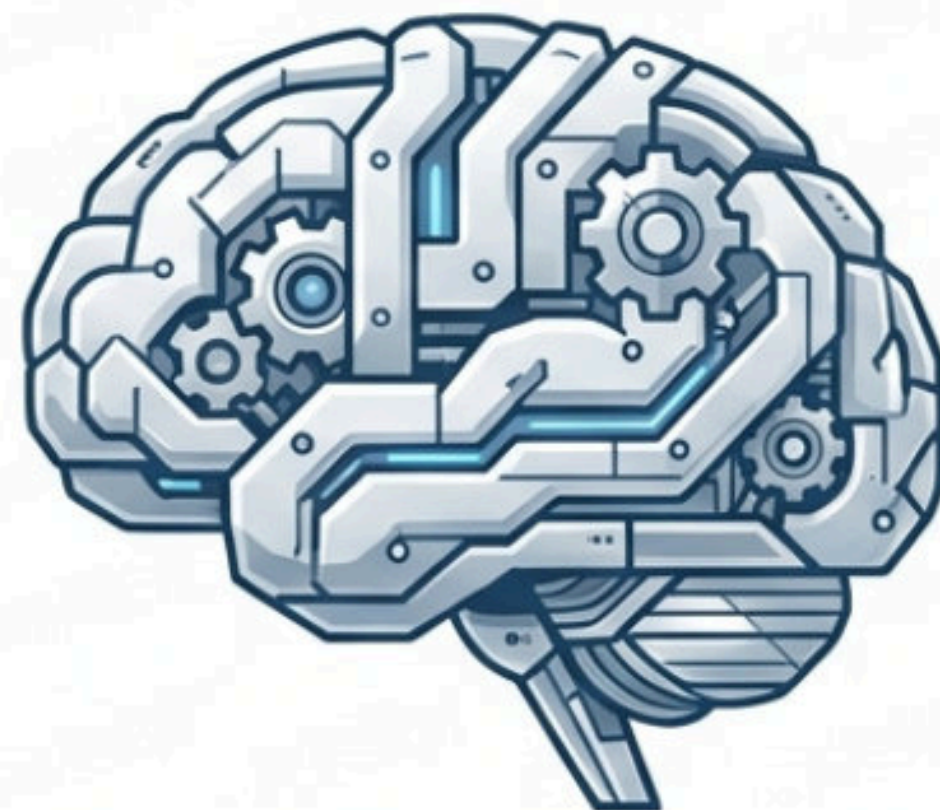
今天是你的工程師測試日。
準備好找出系統的漏洞了嗎？

寫下 100 個「如果...那麼...」的積木，這算是 AI 嗎？



✓ 是 AI (人工智慧)

選邊站！請準備好
告訴工程總監，
你做出這個選擇的
「一個關鍵理由」。



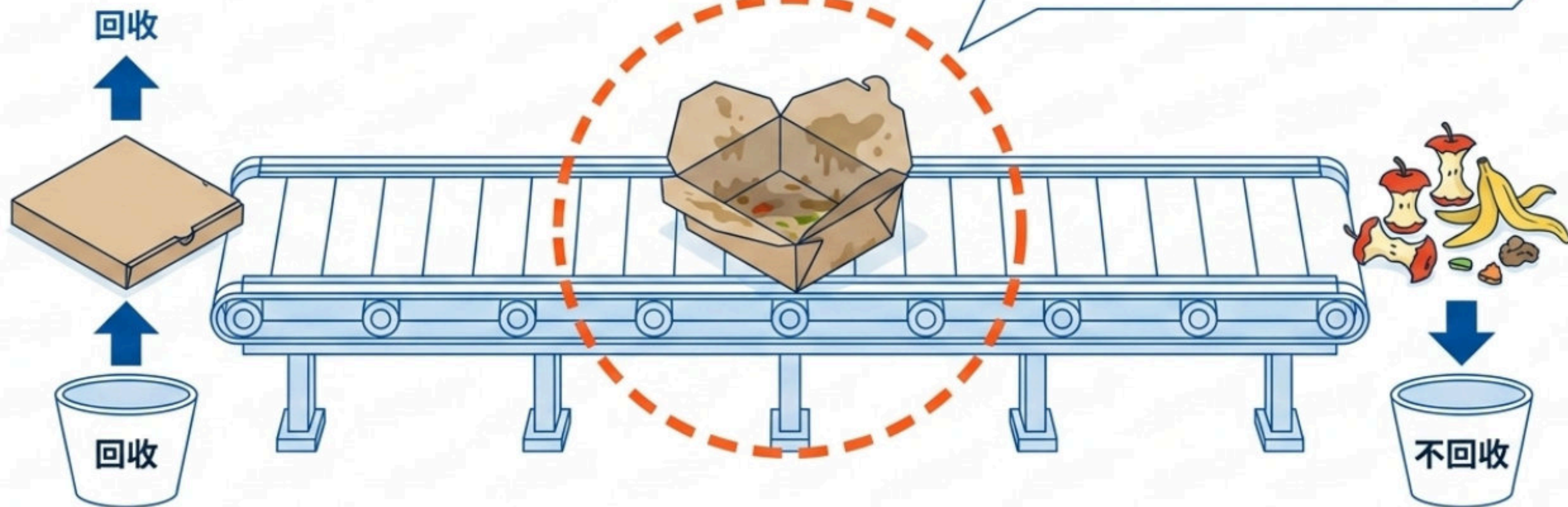
✗ 不是 AI (只是一般程式)

資源回收機的兩難： 它該怎麼處理這項物品？

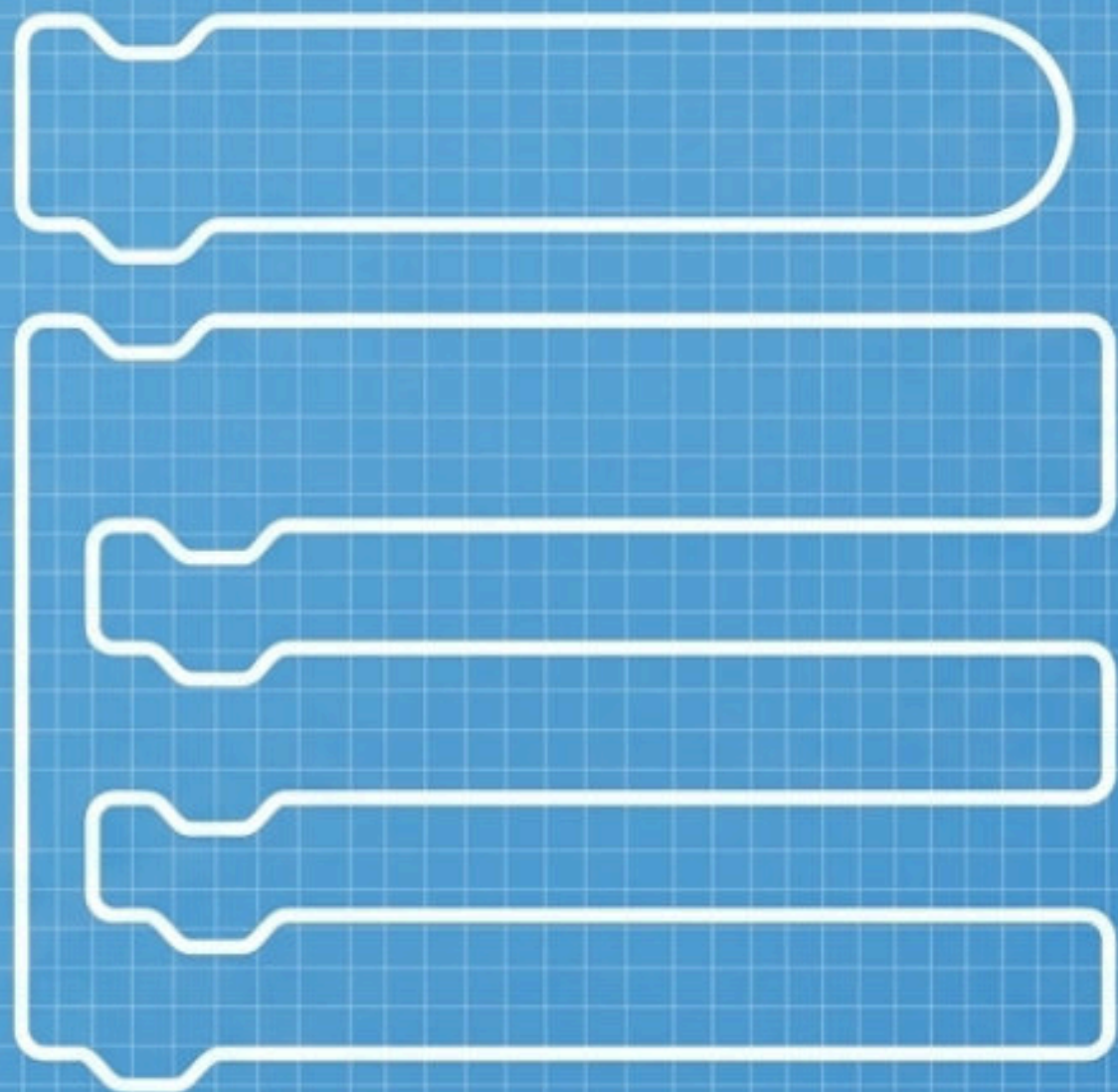
已知系統設定：

- 「紙類」 → 回收
- 「廚餘」 → 不回收

一個沾滿大量油污與剩菜的紙餐盒，機器會怎麼做？
你覺得它「應該」怎麼做？



實作任務：打造第一版測試原型 (v1)



- ☐ 建立兩個角色（互動對象）。
- ☐ 使用「詢問」積木獲取使用者輸入。
- ☐ 使用「如果／否則」完成簡單的二分類（例如：安全／警告，或可回收／不可回收）。



請先專注於讓「最正常」的資料成功分類！

系統崩潰報告：規則的極限與邊界

現在，把這些資料丟進你的 v1 分類器：



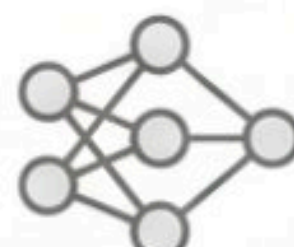
⚠ 輸入：「空白」（什麼都不打）

⚠ 輸入：「🦠👽🚀」（外星文）

⚠ 輸入：「油污紙餐盒」

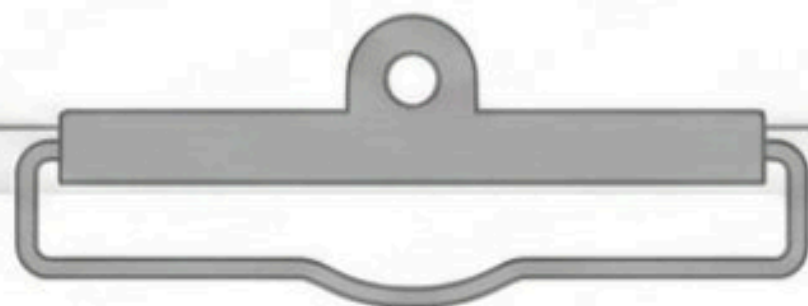
系統崩潰或誤判了嗎？當初寫下這些「如果/否則」死規則的人，究竟是誰？

兩種大腦的對決：規則式 vs. 學習式



	規則式 (Rule-based)	學習式 (ML-based)
誰做決定？	程式設計者（人類）明確寫入條件。	模型從大量標記資料中自己找出規律。
最大優勢	過程透明，完全可以解釋為什麼這樣分類。	能處理人類無法寫出明確規則的複雜樣式（如人臉、語音）。
致命弱點	遇到「規則外」的邊界案例立刻失效。	遇到「訓練資料沒看過」的案例容易嚴重猜錯。

基礎路徑：拆解你的分類規則



1. 用「如果／否則」的句型，明確寫出你的 3 條分類規則。



2. 標示釐清：這些規則是由「誰」決定的？



3. 找碴挑戰：想出 1 筆完全不在這 3 條規則內的「奇怪資料」。

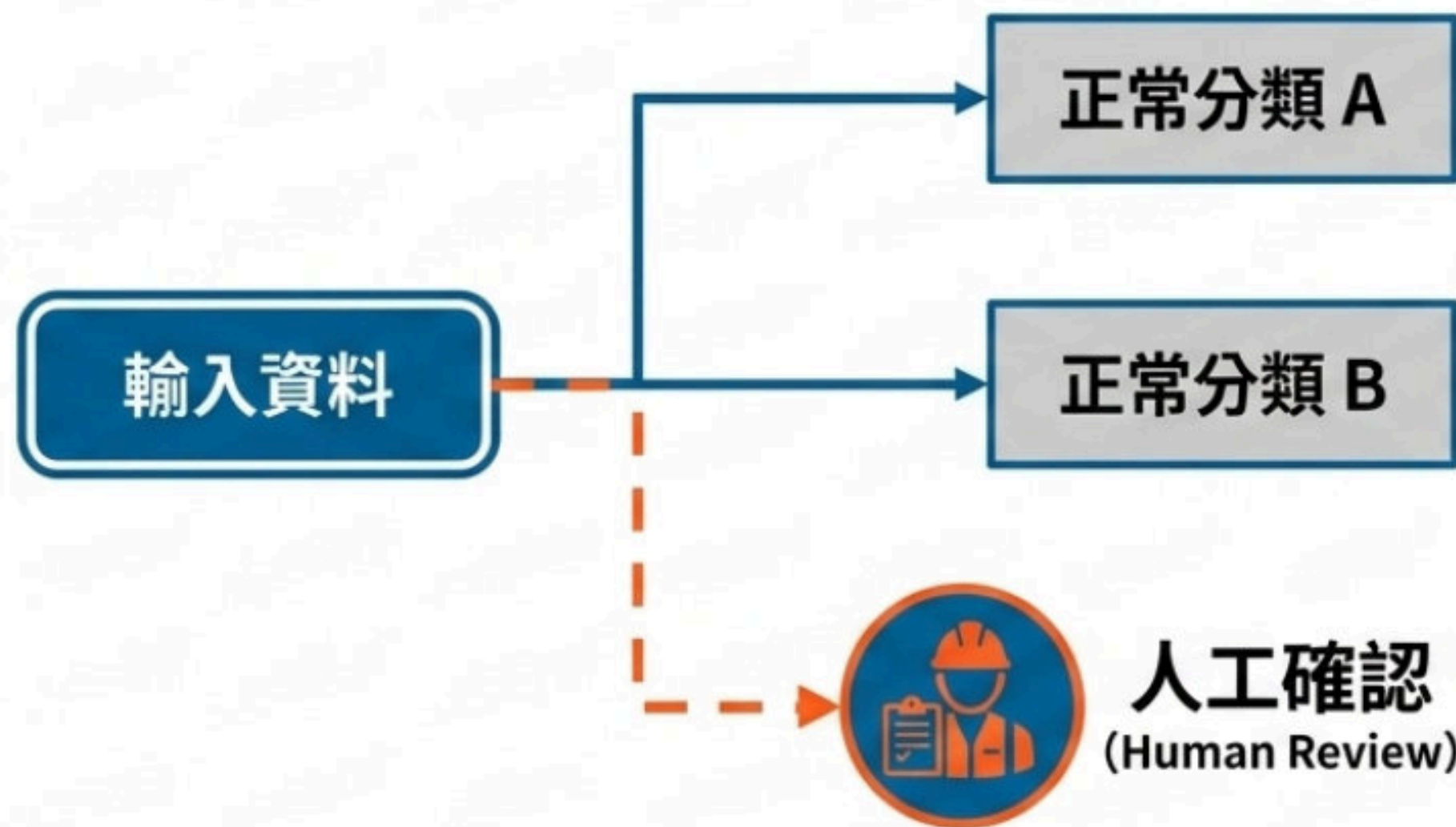
進階路徑：建立分類測試矩陣

不要只測會成功的情境！建立至少 6 筆測試資料來挑戰系統：

✅ 正常輸入 (Normal)	⚠️ 邊界輸入 (Edge)	❌ 錯誤輸入 (Error)
明顯符合規則，系統理應 100% 答對。	模糊地帶，具備兩種分類的特徵，容易引起人類爭議。	亂打字、完全留白、特殊符號。

研究者路徑：設計「不確定」的第三條路

當證據不足時，系統「硬猜」的代價很高。



核心挑戰：你能設計出第三種輸出——
「我不確定，交給人類判斷」嗎？

修改你的 Scratch 程式 (v2)，訂出觸發「人工確認」的具體條件。

案件調查一：主控權在誰手裡？

「規則式分類器」的判斷條件，究竟是由誰決定的？

[A] 程式從大量資料中自己學會的

[B] 程式設計者明確寫入系統的

[C] 使用者在操作時隨機決定的

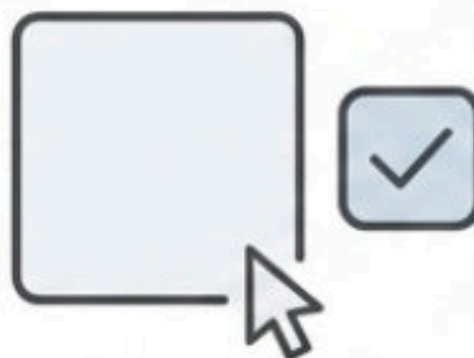
追問：請打開你的 Scratch 程式，指出面上的哪一塊積木是你的「證據」！

案件調查二：尋找最模糊的邊界

下列哪一筆輸入，最像是測試資源回收系統的「邊界案例 (Edge Case)」？



【A】一個完全乾淨的寶特瓶



【B】什麼都不輸入，直接按確認鍵

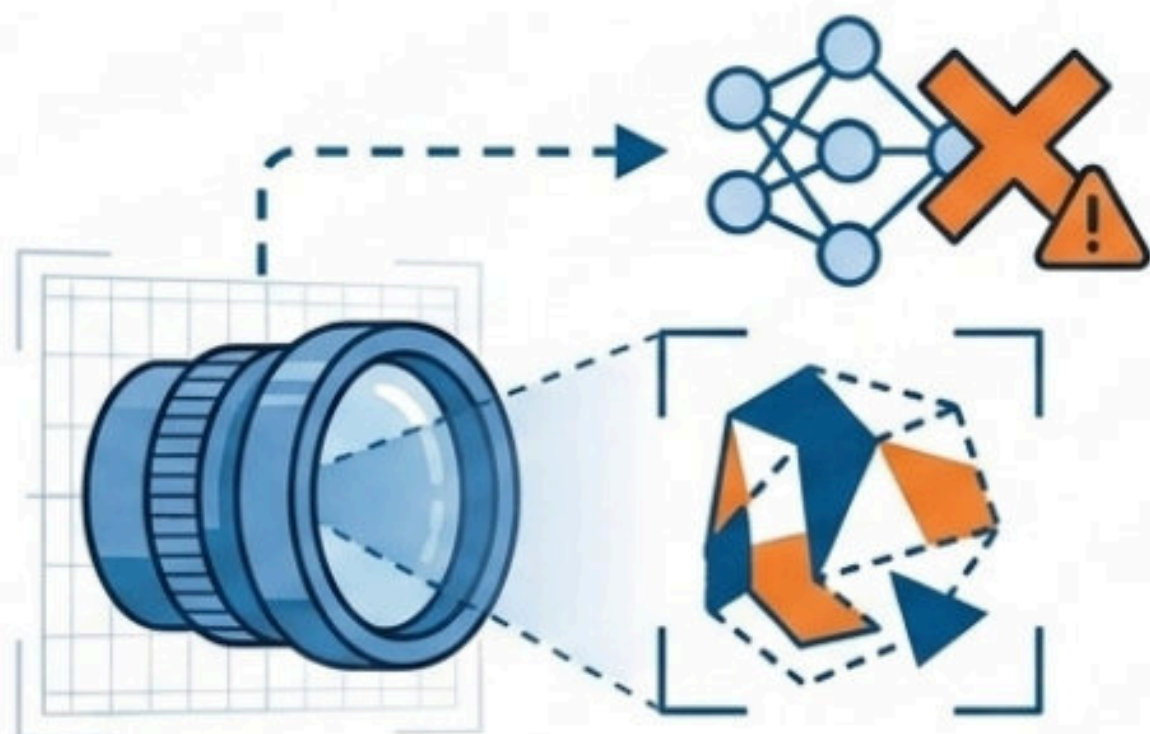


【C】沾有大量油污與剩菜的紙餐盒

追問：為什麼選項 [C] 比選項 [A] 更容易引起人類工程師之間的爭議？

案件調查三：學習式系統的盲點

如果我們用大量圖片教一個「學習式分類器」，但它今天卻嚴重猜錯了，最可能的原因是什麼？



系統診斷：未能識別的輸入 (Unknown Input)

追問：模型只能依賴「看過」的規律。當遇到沒看過的東西時，它缺少了人類大腦的什麼能力？

[A] 訓練資料裡根本沒有類似的案例

[B] Scratch 背景音樂太大聲產生干擾

[C] 程式設計師忘記寫下「如果/否則」

系統結案報告：盤點你的 v2 分類器

請檢視你今天完成的作品，完成以下宣告：

「我的系統現在已經成功學會處理 _____。」

「但經過測試，它目前還無法安全處理 _____。」

「如果我要解決這個漏洞，我下一步的修改計畫會是 _____。」



沒有完美的系統，只有持續進化的邊界



無論是人寫死規則的「規則式」，
還是從資料尋找規律的「學習式」，
分類器永遠不會完美。

真正的 AI 工程師，是透過不斷的「邊界測試」來挑戰系統，讓它在失敗中變得更強大。